191.002 VU Betriebssysteme

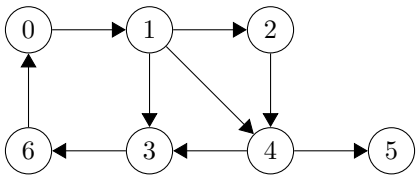
EXERCISE 1

Last update 2026-10-20 (Version 6f3271b3)

Feedback Arc Set

Implement an algorithm which removes cycles in a directed graph by removing the least edges possible. A set of edges which must be removed to make a graph acyclic is also called a feedback arc set; and the set with the least edges is a minimal feedback arc set.

For instance, consider the following directed graph:



This graph has some cycles, such as $\{ 0 \to 1 \to 3 \to 6 \to 0 \}$ or $\{ 0 \to 1 \to 2 \to 4 \to 3 \to 6 \to 0 \}$. All of these cycles can be broken by removing a single edge, for instance by removing the edge $(0, 1)$. Therefore, $\{ (0, 1) \}$ is a minimal feedback arc set for this graph. Note that there are other feedback arc sets of equal length, such as $\{ (6, 0) \}$ or $\{ (3, 6) \}$.

The problem of finding a minimal feedback arc set is NP-complete, therefore an exact solution becomes infeasible for moderately large graph sizes. However, a randomized algorithm can be used to find a good approximation, i.e. to find a feedback arc set which is close to minimal. [1]

A simple randomized algorithm for this problem generates a feedback arc set by executing following steps:

- Order the vertices of the graph randomly, i.e. generate a random permutation of the set of vertices.
- Select all edges (u, v) for which $u > v$ in the ordering. These edges form a feedback arc set.

  For instance, a random permutation of the vertices of the above graph might be $\{ 3, 6, 1, 5, 0, 2, 4 \}$. Consider the edge $(0, 1)$: in the permutation, 1 comes before 0, thus $0 > 1$ in the permutation and therefore the edge is selected. Repeating this for all edges of the graph, we find that the edges for which $u > v$ for this permutation are $\{ (0, 1), (1, 3), (4, 3), (4, 5) \}$. Therefore these edges are one possible feedback arc set for this graph (although not a mimimal one).

  Proof: The subgraph L, which is obtained by removing all edges of this feedback arc set, contains only the edges (u, v) for which $u < v$ in our ordering. Therefore, the permutation generated in the first step is a topological ordering of L, and consequently L must be acyclic. [2]

These steps are executed repeatedly and the smallest feedback arc set so far is retained.

[1] You can read more about the complexity of the feedback arc set problem and about Monte Carlo randomized algorithms in general on Wikipedia:
`https://en.wikipedia.org/wiki/Feedback_arc_set`
`https://en.wikipedia.org/wiki/Monte_Carlo_algorithm`

[2] More background information on topological ordering and directed acyclic graphs can be found on Wikipedia:
`https://en.wikipedia.org/wiki/Topological_sorting`

Although it might seem surprising given its simplicity, this algorithm has an expected runtime which is polynomial in the size of the graph and thus is on average much faster than an algorithm which tries to find an exact solution.

Instructions

In order to further reduce the runtime of this algorithm, multiple processes generate the random feedback arc sets in parallel and report their results to a supervisor process, which remembers the set with the least edges.

Write two programs: a **generator program** and a **supervisor program**. **Multiple generator processes** generate random solutions to the problem and report their solutions to **one supervisor process**. The supervisor process remembers the best solution so far. The processes communicate with eachother by means of a circular buffer, which is implemented using shared semaphores and a shared memory.

Supervisor

The supervisor sets up the shared memory and the semaphores and initializes the circular buffer required for the communication with the generators. It then waits for the generators to write solutions to the circular buffer.

```
SYNOPSIS:
    supervisor [-h] [-o outfile] [-n limit] [-w delay]
```

If the option `-h` is given, the program shall print usage information and terminate with exit status `EXIT_SUCCESS`.

If the option `-o` is given, the output is also written to the specified file (`outfile`).

The argument `limit` specifies a limit (integer value) for the number of generated solutions. If `limit` is omitted, it should be considered as infinite.

The argument `delay` specifies a delay (in seconds) before reading the first solution from the buffer. If `delay` is omitted, it should be considered as zero.

The program shall print usage and exit with `EXIT_FAILURE` whenever the command line contains unrecognized or badly formatted options, e.g., `supervisor -x` should be considered a user error.

Once initialization is complete, it waits for `delay` seconds[3], and then the supervisor reads the solutions from the circular buffer and remembers the best solution so far, i.e. the solution with the least edges. Every time a better solution than the previous best solution is found, the supervisor writes the new solution to *stderr*. If a generator writes a solution with 0 edges to the circular buffer, then the graph is acyclic and the supervisor terminates. In this case, the supervisor prints `The graph is acyclic!` to *stdout*.

Otherwise the supervisor keeps reading results from the circular buffer until it receives a `SIGINT`, a `SIGTERM` signal, or the number of read solutions from the circular buffer exceeds the value `limit`. In this case, the supervisor prints `The graph might not be acyclic, best solution removes N edges.` to *stdout* (where `N` is replaced with the number of edges of the best found solution).

Before terminating, the supervisor notifies all generators that they should terminate as well. This can be done by setting a variable in the shared memory, which is checked by the generator processes before writing to the buffer. The supervisor then unlinks all shared resources and exits.

[3]Only for this purpose, you may use the function `sleep()`. This delay is intended to give enough time to start multiple generators on the command line manually.

Generator

The generator program takes a graph as input. The program repeatedly generates a random solution to the problem as described on the first page and writes its result to the circular buffer. It repeats this procedure until it is notified by the supervisor to terminate.

The generator program takes as arguments the set of edges of the graph:

```
SYNOPSIS
    generator EDGE1...
EXAMPLE
    generator 0-1 1-2 1-3 1-4 2-4 3-6 4-3 4-5 6-0
```

Each positional argument is one edge; at least one edge must be given. An edge is specified by a string, with the indices of the two nodes it connects separated by a `-`. Since the graph is directed, the edge starts at the first node and leads to the second node. Note that the number of nodes of the graph is implicitly provided through the indices in the edges. The generator must not make any assumptions as to the number of edges that may be specified and programmatically accept a graph of any size (as well in the number edges as in the number of nodes) [4]. In the example above the generator program is called with the graph shown on the first page.

The generator uses the algorithm described on the first page to generate random feedback arc sets for the given graph. It writes these feedback arc sets to the circular buffer, one at a time; therefore a feedback arc set is a single element of the circular buffer. The generator may produce debug output, describing the feedback arc sets which it writes to the circular buffer (only if the compile flag `-DDEBUG` is set).

[4] Practically, the number of edges which can be specified is limited by the system constant `ARG_MAX`. However, your program must not make any assumptions based on this (or other) limits.

Circular Buffer

The generators report their solutions to the supervisor by means of a circular buffer. The generators write their solutions to the write end of the circular buffer and the supervisor reads them from the read end.

A circular buffer is a common data structure which uses a single fixed-size buffer to implement a queue (i.e. a FIFO buffer). A circular buffer is essentially **an array of the elements** you want to pass through. Elements are written successively to the array and once the end of the array is reached, writing restarts from the beginning. Similarly, elements are also read successively from the array, restarting from the beginning upon reaching the end, thus reading the elements in the exact same order they have been written. [5]

Implement your circular buffer such that **reading and writing** to the buffer can happen **simultaneously**. Some synchronization is required in order to avoid overwriting data which has not been read yet and also to avoid trying to read from locations which have not been written yet. This is achieved using two semaphores: **one semaphore tracking the free space** in the circular buffer and **one semaphore tracking the used space**. The value of the free space semaphore corresponds to the amount of free space in the buffer array; it is initialized to the size of the buffer, since initially the circular buffer is empty. The value of the used space semaphore corresponds to the amount of used space in the buffer array; it is initialized to 0.

Upon **writing to the circular buffer**, the free space semaphore is decremented; if the buffer is currently full and there is no free space to write to, this intentionally blocks the write until space becomes available. After the write is complete, the used space semaphore is incremented, since the buffer now holds one more element which can be read.

Upon **reading from the circular buffer**, the used space semaphore is decremented; if the buffer is currently empty and there is no data to be read, this intentionally blocks the read until data becomes available. After the read is complete, the free space semaphore is incremented, since the position occupied by the element which has been read just has been freed up.

Since multiple generators write to the circular buffer, an additional semaphore will be required to guarantee a **mutually exclusive access to the write end** of the circular buffer.

Hints

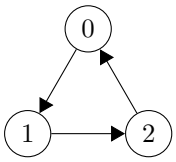
- Store the graph in a way which is suitable for the implementation of the randomized algorithm.
- If you need inspiration for generating random permutations of the graph's vertices, you might want to have a look at the Fisher-Yates shuffle algorithm. [6]
- Think of a suitable structure to store a feedback arc set in the circular buffer. Choose a limit on the maximum number of edges which it can contain. Since we are looking for small solutions, the generator can discard any solutions which contain too much edges instead of writing them to the circular buffer. You may choose a limit as low as 8 edges.
- Select a reasonable size for the circular buffer. If the size is too small, then the generators will spend a lot of time waiting for free space to become available. If the size is too large, the circular buffer is wasting resources. The size of the shared memory should not exceed 4 KiB.

[5] Visualizations of circular buffers can be found on Wikipedia:
`https://en.wikipedia.org/wiki/Circular_buffer`

[6] A description of this algorithm along with a pseudocode example can be found on Wikipedia:
`https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle`
If you want to read more about algorithms to generate random permutations, have a look at:
`https://en.wikipedia.org/wiki/Permutation#Algorithms_to_generate_permutations`

Examples

Simple cyclic graph



The minimal feedback arc set of this graph contains 1 edge; in fact there are 3 minimal feedback arc sets:

$\{ (0,1) \}$, $\{ (1,2) \}$ and $\{ (2,0) \}$

Invocation of the supervisor:

```
$ ./supervisor -n 3000
Solution with 2 edges: 1-2 2-0
Solution with 1 edges: 2-0
The graph might not be acyclic,
   best solution removes 1 edges.
```

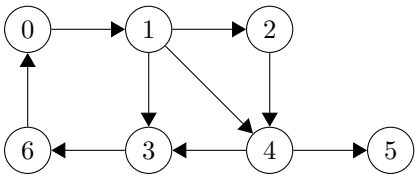
Invocation of one generator:

```
$ ./generator 0-1 1-2 2-0
```

Invocation of 10 generators in parallel:

```
$ for i in {1..10}; do (./generator 0-1 1-2 2-0 &); done
```

Graph from the first page



This graph has again 3 minimal feedback arc sets, each with one edge:

$\{ (0,1) \}$, $\{ (3,6) \}$ and $\{ (6,0) \}$

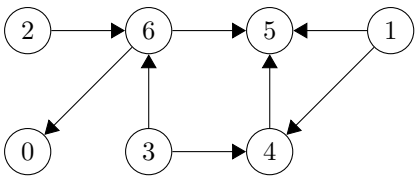
Invocation of the supervisor:

```
$ ./supervisor -n 3000
Solution with 5 edges: 1-2 1-3 1-4 2-4 6-0
Solution with 4 edges: 0-1 2-4 4-3 6-0
Solution with 3 edges: 0-1 1-4 2-4
Solution with 1 edges: 0-1
The graph might not be acyclic,
   best solution removes 1 edges.
```

Invocation of the generator:

```
$ ./generator 0-1 1-2 1-3 1-4 2-4 3-6 4-3 4-5 6-0
```

Acyclic graph



This graph is already acyclic.

Invocation of the supervisor:

```
$ ./supervisor
Solution with 5 edges: 1-4 1-5 2-6 3-4 3-6
Solution with 2 edges: 3-6 4-5
Solution with 1 edges: 6-0
The graph is acyclic!
```

Invocation of the generator:

```
$ ./generator 1-4 1-5 2-6 3-4 3-6 4-5 6-0 6-5
```

If you want to test your implementation with a challenging graph, try this one:

```
0-2 0-9 0-11 1-4 3-2 3-6 4-2 4-9 5-2 5-11 6-2 6-4 7-2 7-4 7-5 7-8 7-16 7-17 8-9 8-12
8-17 10-2 10-9 11-2 12-1 12-6 12-10 13-5 13-6 13-8 14-4 14-12 15-8 15-11 15-13 16-1
16-6 16-17 17-6 17-10 17-11 18-7 18-8 18-11
```

Note: This graph is acyclic, therefore one of your generators should eventually come up with a solution with 0 edges.

Mandatory testcases

Input is shown in blue color. Output to *stdout* (and *stderr*) is shown in black. (Note that in the following output sections `EXIT_SUCCESS` equals `0`, and `EXIT_FAILURE` equals `1`. Refer to `stdlib.h` for further details.) `^C` indicates `CTRL+C`, `^D` indicates `CTRL+D`. The placeholder `<usage message>` must be replaced by a proper usage message (printed to *stdout*), `<error message>` must be replaced by a meaningful error message (which is printed to *stderr*).

Testcase 01: simple-1

Start your supervisor first.

```
$ ./supervisor
The graph is acyclic!
$ echo $?
0
```

Then start the generator.

```
$ ./generator 0-1 0-2 2-1
$ echo $?
0
```

Testcase 02: simple-2

Start your supervisor first.

```
$ ./supervisor -n 500
The graph might not be acyclic, best solution \
     removes 1 edges.
$ echo $?
0
```

Then start the generator.

```
$ ./generator 0-1 1-2 2-0
$ echo $?
0
```

Testcase 03: multi-1

Start your supervisor first.

```
$ ./supervisor -n 100000 -w 2
The graph might not be acyclic, best solution \
     removes 1 edges.
$ echo $?
0
```

Then start the generators.

```
$ for i in `seq 0 2`; do ( ./generator 0-1 1-2 \
     1-3 1-4 2-4 3-6 4-3 4-5 6-0 ; echo \
     "generator returned $?" ) & done;
generator returned 0
generator returned 0
generator returned 0
```

Testcase 04: multi-2

Start your supervisor first.

```
$ ./supervisor -n 100000 -w 2
The graph is acyclic!
$ echo $?
0
```

Then start the generators.

```
$ for i in `seq 0 2`; do ( ./generator 1-4 1-5 \
     2-6 3-4 3-6 4-5 6-0 6-5 ; echo "generator \
     returned $?" ) & done;
generator returned 0
generator returned 0
generator returned 0
```

Testcase 05: multi-3

Start your supervisor first.

```
$ ./supervisor -n 1000000 -w 2
The graph is acyclic!
$ echo $?
0
```

Then start the generators.

```
$ for i in `seq 0 2`; do ( ./generator 0-8 0-9 \
    0-15 1-7 1-10 1-15 2-5 2-13 2-15 3-6 3-14 \
    3-15 4-11 4-12 4-15 5-6 5-12 6-11 7-8 7-13 \
    8-14 ; echo "generator returned $?" ) & done;
generator returned 0
generator returned 0
generator returned 0
```

Testcase 06: multi-4

Start your supervisor first.

```
$ ./supervisor -n 1000000 -w 2
The graph is acyclic!
$ echo $?
0
```

Then start the generators.

```
$ for i in `seq 0 2`; do ( ./generator 0-1 0-2 \
    1-13 1-14 1-16 2-11 2-12 2-15 3-4 3-5 3-9 \
    3-10 4-6 4-7 4-8 5-6 5-11 5-12 6-13 6-14 \
    7-9 7-13 7-16 8-10 8-14 8-16 9-12 9-15 ; \
    echo "generator returned $?" ) & done;
generator returned 0
generator returned 0
generator returned 0
```

Testcase 07: multi-5

Start your supervisor first.

```
$ ./supervisor -n 100000 -w 2
The graph might not be acyclic, best solution \
    removes 2 edges.
$ echo $?
0
```

Then start the generators.

```
$ for i in `seq 0 4`; do ( ./generator 0-1 1-2 \
    2-3 3-4 4-5 5-0 6-7 7-8 8-6 ; echo \
    "generator returned $?" ) & done;
generator returned 0
generator returned 0
generator returned 0
generator returned 0
generator returned 0
```

Testcase 10: no-supervisor

```
$ ./generator 0-1 0-2 1-2
<error message>
$ echo $?
1
```

Testcase 11: usage

```
$ ./supervisor -x
<usage message>
$ echo $?
1
```

Testcase 12: usage-2

```
$ ./supervisor -h
<usage message>
$ echo $?
0
```

Testcase 13: file-1

Start your supervisor first.

```
$ ./supervisor -o log.txt
The graph is acyclic!
$ echo $?
0
$ cat log.txt
The graph is acyclic!
```

Then start the generator.

```
$ ./generator 0-1 0-2 2-1
$ echo $?
0
```

Testcase 14: file-2

```
$ touch existingTestfile
$ chmod 0000 existingTestfile
$ echo "test" > existingTestfile
bash: existingTestfile: Permission denied
$ ./supervisor -o log.txt
<error message>
$ echo $?
1
```

Coding Recommendations and Guidelines

Your score depends upon the compliance of your submission to the presented guidelines and recommendations. Violations result in deductions of points. Hence, before submitting your solution, go through the following list and check if your program complies.

Rules

Compliance with these rules is essential to get any points for your submission. A violation of any of the following rules results in 0 points for your submission.

1. All source files of your program(s) must compile via
   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
                                  -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```
   without *errors* and your program(s) must link without *errors*. The compilation flags must be used in the Makefile. The feature test macros must not be bypassed (i.e., by undefining these macros or adding some in the C source code).
2. The functionality of the program(s) must conform exactly to the assignment. The program(s) shall operate according to the specification/assignment given the test cases in the respective assignment. Additional white spaces or any other deviation from the specified input and output format may lead to a failure of the respective test case.

General Guidelines

Violation of following guidelines leads to a deduction of points.

1. All source files of your program(s) must compile with
   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
                                  -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```
   without *warnings and info messages* and your program(s) must link without warnings.
2. There must be a Makefile implementing the targets: `all` to build the program(s) (i.e. generate executables) from the sources (this must be the first target in the Makefile); `clean` to delete all files that can be built from your sources with the Makefile.
3. All targets of your Makefile must be idempotent. I.e. execution of `make clean; make clean` must yield the same result as `make clean`, and must not fail with an error.
4. The program shall operate according to the specification/assignment without major issues (e.g., segmentation fault, memory corruption).
5. Arguments have to be parsed according to UNIX conventions (we strongly encourage the use of `getopt(3)`). The program has to conform to the given synopsis/usage in the assignment. If the synopsis is violated (e.g., unspecified options or too many arguments), the program has to terminate with the usage message containing the program name and the correct calling syntax. Argument handling should also be implemented for programs without arguments.
6. Correct (=normal) termination, including a cleanup of resources.
7. Upon success the program has to terminate with exit code 0, in case of errors with an exit code greater than 0. We recommend to use the macros `EXIT_SUCCESS` and `EXIT_FAILURE` (defined in `stdlib.h`) to enable portability of the program.

8. If a function indicates an error with its return value, it *should* be checked in general. If the subsequent code depends on the successful execution of a function (e.g. resource allocation), then the return value *must* be checked.

9. Functions that do not take any parameters have to be declared with `void` in the signature, e.g., `int get_random_int(void);`.

10. Procedures (i.e., functions that do not return a value) have to be declared as `void`.

11. Error messages shall be written to `stderr` and should contain the program name `argv[0]`.

12. It is forbidden to use the functions: `gets`, `scanf`, `fscanf`, `atoi` and `atol` to avoid crashes due to invalid inputs.

FORBIDDEN	USE INSTEAD
gets	fgets
scanf	fgets, sscanf
fscanf	fgets, sscanf
atoi	strtol
atol	strtol

13. Documenation is mandatory. Format the documentation in Doxygen style (see Wiki and Doxygen's intro).

14. Write meaningful comments. For example, meaningful comments describe the algorithm, or why a particular solution has been chosen, if there seems to be an easier solution at a first glance. Avoid comments that just repeat the code itself
(e.g., `i = i + 1; /* i is incremented by one */`).

15. The documentation of a module must include: name of the module, name and student id of the author (`@author` tag), purpose of the module (`@brief`, `@details` tags) and creation date of the module (`@date` tag).

    Also the Makefile has to include a header, with author and program name at least.

16. Each function shall be documented either before the declaration or the implementation. It should include purpose (`@brief`, `@details` tags), description of parameters and return value (`@param`, `@return` tags) and description of global variables the function uses (`@details` tag).

    You should also document `static` functions (see `EXTRACT_STATIC` in the file `Doxyfile`). Document visible/exported functions in the header file and local (`static`) functions in the C file. Document variables, constants and types (especially `structs`) too.

17. Documentation, names of variables and constants shall be in English.

18. Internal functions shall be marked with the `static` qualifier and are not allowed to be exported (e.g., in a header file). Only functions that are used by other modules shall be declared in the header file.

19. All exercises shall be solved with functions of the C standard library. If a required function is not available in the standard library, you can use other (external) functions too. Avoid reinventing the wheel (e.g., re-implementation of `strcmp`).

20. Name of constants shall be written in upper case, names of variables in lower case (maybe with first letter capital).

21. Use meaningful variable and constant names (e.g., also semaphores and shared memories).

22. Avoid using global variables as far as possible.

23. All boundaries shall be defined as constants (macros). Avoid arbitrary boundaries. If boundaries are necessary, treat its crossing.

24. Avoid side effects with `&&` and `||`, e.g., write `if(b != 0) c = a/b;` instead of `if(b != 0 && c = a/b)`.

25. Each `switch` block must contain a `default` case. If the case is not reachable, write `assert(0)` to this case (defensive programming).

26. Logical values shall be treated with logical operators, numerical values with arithmetic operators (e.g., test 2 strings for equality by `strcmp(...) == 0` instead of `!strcmp(...)`).

27. Indent your source code consistently (there are tools for that purpose, e.g., `indent`).

28. Avoid tricky arithmetic statements. Programs are written once, but read more times. Your program is not better if it is shorter!

29. For all I/O operations (read/write from/to `stdin`, `stdout`, files, sockets, pipes, etc.) use *either* standard I/O functions (`fdopen(3)`, `fopen(3)`, `fgets(3)`, etc.) *or* POSIX functions (`open(2)`, `read(2)`, `write(2)`, etc.). Remember, standard I/O functions are buffered. Mixing standard I/O functions and POSIX functions to access a common file descriptor can lead to undefined behaviour and is therefore forbidden.

30. If asked in the assignment, you must implement signal handling (`SIGINT`, `SIGTERM`). You must only use *async-signal-safe* functions in your signal handlers.

31. Close files, free dynamically allocated memory, and remove resources after usage.

32. Don't waste resources due to inconvenient programming. Header files shall not include implementation parts (exception: macros).

33. To comply with the given testcases, the program output must exactly match the given specification. Therefore you are only allowed to to print any debug information if the compile flag `-DDEBUG` is set.

Exercise 1 Guidelines

Violation of following guidelines leads to a deduction of points in exercise 1b.

1. Correct use of named sempahores (`sem_open(3)`, `sem_close(3)` `sem_unlink(3)`) and POSIX shared memory (`shm_overview(7)`) for inter-process communication of separated programs (e.g., server and client).

   Use your matriculation number as prefix in the names of all resources.

2. "Busy waiting" is forbidden. (Busy waiting is the repeated check of a condition in a loop for synchronization purposes.)

3. Synchronization with `sleep` is forbidden.